

PRAKTIKUM SISTEM OPERASI

**MODUL XI
MEMORI XINU**



Disusun Oleh :

Alya Rabani / 2311104076

SE-07-02

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs.

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025

JURNAL

1. Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

```
#include <xinu.h>

// Nama: Alya Rabani
// NIM: 2311104076

shellcmd xsh_freememory(int nargs, char *args[]) {

    struct memblk *blk;
    uint32 totalFreeSpace = 0;

    kprintf("Free List:\n");
    kprintf("Block address Length (dec) Length (hex)\n");
    kprintf("-----\n");

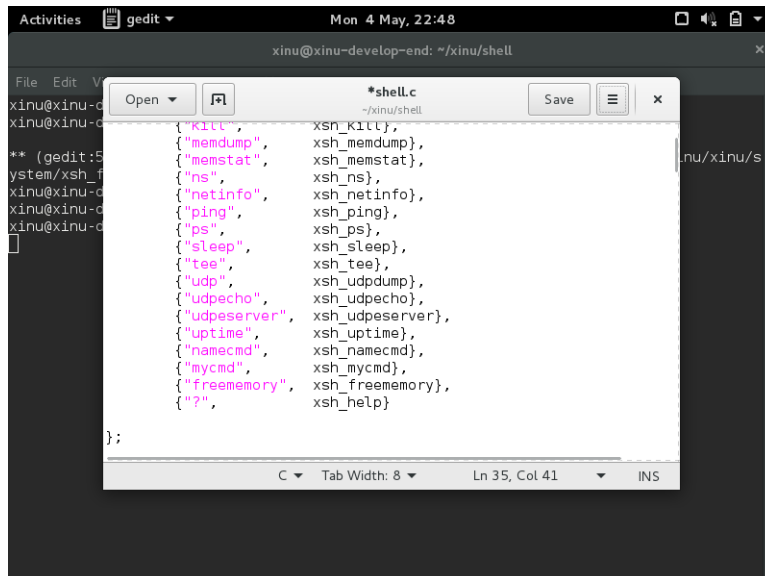
    blk = memlist.mnext;

    while (blk != NULL) {
        kprintf("0x%08X %10d 0x%08X\n", (uint32) blk, blk->mlength, blk->mlength);
        totalFreeSpace += blk->mlength;
        blk = blk->mnext;
    }

    kprintf("\nTotal FreeSpace %d\n", totalFreeSpace);

    return 0;
}
```

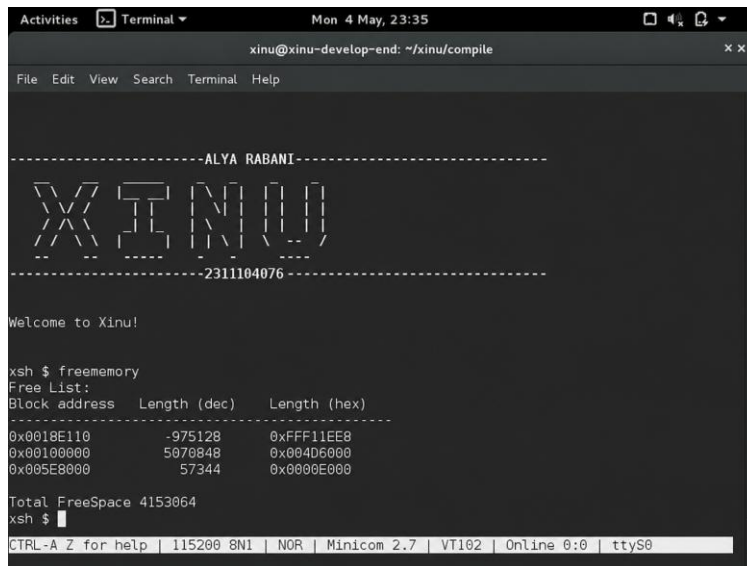
Edit shell.c



Edit shprototypes.h

```
/* in file xsh_freememory.c */
extern shellcmd xsh_freememory(int nargs, char *args[]);
```

Output



```
Activities Terminal Mon 4 May, 23:35
xinu@xinu-develop-end: ~/xinu/compile

-----ALYA RABANI-----
XINU
-----2311104076-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018E110      -975128    0xFFFF1EE8
0x00100000       5070848    0x004D6000
0x005E8000       57344      0x0000E000

Total FreeSpace 4153064
xsh $
```

2. Jawablah pertanyaan berikut:

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Jawab: Pemisahan antara Data Segment (inisialisasi) dan BSS Segment (Block Started by Symbol / tidak diinisialisasi) dilakukan untuk efisiensi ukuran berkas biner (executable file) dan kecepatan proses loading.

Data Segment: Menyimpan variabel global dan statis yang memiliki nilai awal eksplisit (misal: `int x = 10;`). Nilai-nilai ini harus disimpan di dalam disk atau memori flash agar bisa dimuat saat sistem dinyalakan.

BSS Segment: Menyimpan variabel global dan statis yang belum diinisialisasi atau bernilai nol (misal: `int y[1000];`). Karena nilainya pasti nol, Xinu tidak perlu menyimpan ribuan angka nol di dalam berkas biner. Sistem cukup mencatat ukuran total segmen BSS. Saat sistem melakukan booting, Xinu hanya perlu mengalokasikan ruang tersebut di RAM dan membersihkannya (mengisi dengan nol) dalam satu operasi cepat.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Jawab: Ukuran dan kondisi free space (ruang memori yang tersedia) berubah secara dinamis akibat dua operasi utama:

Alokasi (`getmem` / `getstk`): Mengambil sejumlah blok memori dari daftar free space untuk digunakan oleh proses atau heap. Operasi ini langsung mengurangi ukuran total free space dan berpotensi memecah satu blok kosong besar menjadi blok-blok kecil.

Dealokasi (`freemem` / `freestk`): Mengembalikan memori yang sudah tidak digunakan kembali ke daftar free space. Operasi ini menambah ukuran total free space.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Jawab: Pada heap, jika programmer lupa melepaskan memori (`freemem`), akan terjadi memory leak yang lambat laun menghabiskan RAM. Selain itu, alokasi dan dealokasi yang acak memicu fragmentasi eksternal, di mana total memori bebas sebenarnya cukup, namun tersebar dalam potongan-potongan kecil yang tidak saling menyambung, sehingga tidak bisa digunakan untuk alokasi besar.

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Jawab: Xinu menggunakan struktur singly linked list (yang sering disebut sebagai free list) karena karakteristiknya yang sangat cocok untuk keterbatasan memori sistem tertanam (embedded systems):

Tanpa Overhead Memori Tambahan: Xinu menyimpan informasi pointer (next) dan ukuran blok (length) langsung di dalam blok kosong itu sendiri (menggunakan struktur data penampung saat kosong). Jadi, list ini tidak memakan ruang RAM tambahan di luar memori yang memang sedang bebas.

Fleksibilitas Ukuran: Blok memori yang bebas memiliki ukuran yang bervariasi. Linked list mempermudah Xinu untuk menelusuri (traversal), memecah (splitting) satu blok saat dialokasi, atau menyisipkan kembali blok baru secara dinamis berdasarkan alamat memorinya.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawab: Sebagai sistem operasi yang minimalis dan dirancang untuk instruksi/pembelajaran serta perangkat tertanam, tantangan utama manajemen heap di Xinu meliputi:

Tidak Ada Garbage Collection otomatis: Tanggung jawab penuh ada pada programmer. Kelalaian kecil langsung berdampak pada stabilitas sistem.

Fragmentasi Memori yang Agresif: Karena algoritma alokasi Xinu umumnya sederhana (seperti First-Fit berdasarkan urutan alamat), heap sangat rentan mengalami fragmentasi jika aplikasi sering melakukan alokasi dan dealokasi dalam ukuran yang bervariasi.

Tidak Ada Virtual Memory / MMU Support: Xinu bekerja dengan alamat fisik langsung (physical addressing). Tidak ada pemetaan memori virtual untuk merapikan fragmentasi di latar belakang. Jika memori fisik terfragmentasi, sistem tidak bisa "menggeser" isi memori untuk menyatukan ruang kosong tanpa merusak pointer yang sedang digunakan oleh proses aktif.